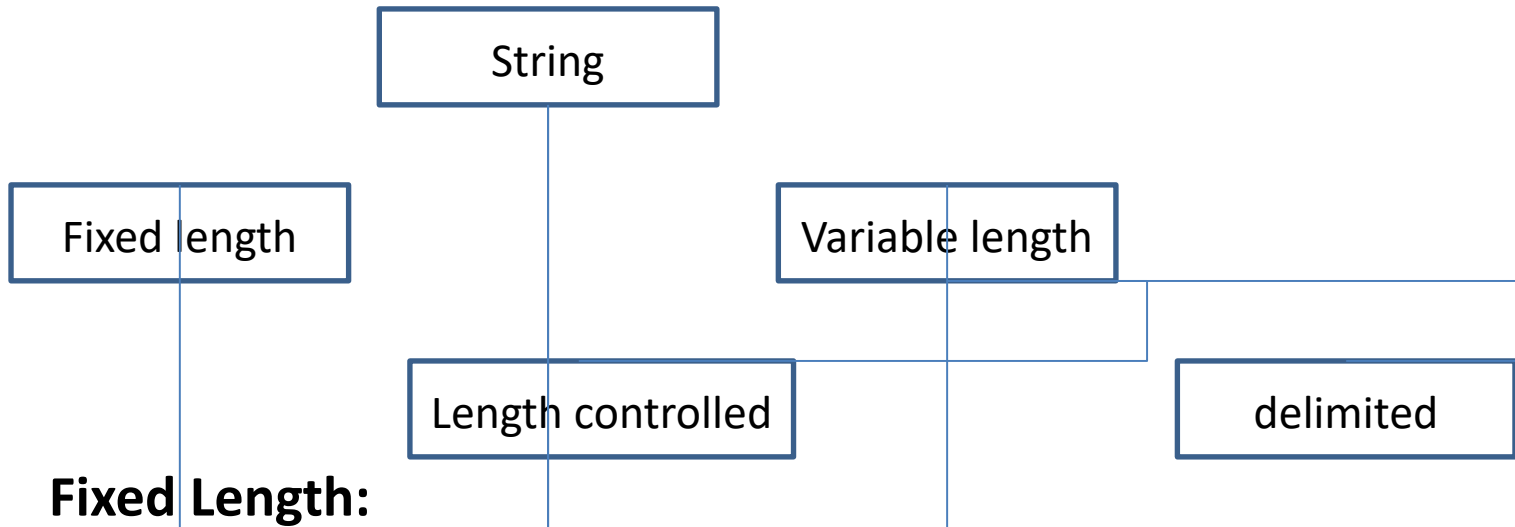

Chapter 6 : String

String-Concept

A string is a series of character treated as a unit.

String Taxonomy and Representation



- The string size is fixed.
- Represented as an array of character

Disadvantage

- If we make the size small, we are unable to store all the data: if we make the size too big, we waste memory.
- A fixed length string must allow to differentiate data and non data character. Character selected to represent non data can not be used as data.

String Taxonomy and...

Variable-length String

Variable size. Can expand or contract to accommodate the data.

Based on the way the end of data represented, there are two types:

- Delimited
- Length-controlled

Delimited String

Delimited String

- The string is represented as **an array of characters** delimited by a special character.
- C++ uses the **ASCII null character(ASCII code 0)**, represented as **escape sequence '\0'**, as a **string delimiter**.
- The string can store up to size -1 characters where size is the size of the array. One byte is left for the delimiter
- The delimited string is also called **C-string style**.

C-string in memory

G	i	r	m	a		D	a	w	i	t	\0							
---	---	---	---	---	--	---	---	---	---	---	----	--	--	--	--	--	--	--

Declaration of C++-string

C-string are implemented using an array of character. Thus it must be declared as an array of character

Syntax:

```
char str_name[size];
```

str_name string can hold up to size – 1 data characters.

Example

```
char studentName[31];
```

Note: Because C-string is represented as an array of character, it will suffer all the limitation of an array.

Initialization of C++-string

C-string can be initialized in two ways:

- **Using the usual array initialization**

Syntax:

```
char str_name[size]={charVal1, charVal2...charVallast, '\0'};
```

Example

```
char studentName[31]={‘G’, ‘i’, ‘r’, ‘m’, ‘a’, ‘ ’, ‘D’, ‘a’, ‘w’, ‘i’, ‘t’ ‘\0’};
```

- **Using string literal**

Syntax:

```
char str_name[size]=string_literal;
```

Example

```
char studentName[31]="Girma Dawit";
```

Note: The length of the string at least should be one less from the size of the array

Reading and Writing C++-string

The one place where C++ allows aggregate operations on arrays is the input and output of C-strings (that is, character arrays)

Reading

Syntax: `cin >> strVar;`

cin is not capable of receiving **multi-word string** such as “hello world”.

This is because the cin “>>” operator considers a space to be a terminating character. Thus it will read strings consisting of single word, but anything typed after a space is thrown away.

To read strings with blanks, use get:

- `cin.get(cstr, m+1);`

Stores the next m characters into cstr but the newline character is not stored in cstr. If the input string has fewer than m characters, the reading stops at the newline character

- `cin.getline(cstr, n, ch);` // read to a maximum of n chrs or up to character ch which ever occur first.
- `cin.getline(cstr, n);` //read to end-of-line

Reading...

Writing

Syntax: `cout << str;`

outputs the content of str on the screen

- `<<` continues to write the contents of str until it finds the null character
- If str does not contain the null character, then we will see strange output. `<<` continues to output data from memory adjacent to name until `'\0'` is found. Your program may crash if it interferes with other programs memory.

Reading...

```
#include <iostream.h>
int main ()
{
    char mystr1[50], mystr2[50];
    cout << "What's your name? ";
    cin >> mystr1
    cout << "Hello " << mystr1 << ".\n"; //if the user enters "Dawit Girma" , output?
    cout << "What's your name? ";
    cin.getline (mystr2, 49);
    cout << "Hello " << mystr2 << ".\n"; //if the user enters "Dawit Girma" , output?
    return 0;
}
```

C++ string Manipulation

Aggregate operations, such as assignment and comparison, are not allowed on C-string.

Example

```
studentName="Abebe Dawit"; //Invalid!!
```

```
studentName=="Abebe Dawit"; //Invalid!!
```

Instead we use C standard string library function.

C-String Library Functions (Require the Header File "string.h")

String Copy Function

strcpy() function: used to copy one string to another.

Syntax: strcpy(destination, source);

Example:

```
strcpy(studentName, "Mesfin Girma" );
```

Caution: if the destination array being too small to hold the information, the copy function destroys the content of adjacent memory causing the program to abort, or running with corrupted data.

To avoid the above problem use the strncpy() function

Strncpy() function: Copies at most n characters of source to destination. If destination has fewer than n characters, it pads with '\0's.

Syntax: strncpy(destination, source, n)

String Concatenation Function

`strcat()` function: appends source to the end of the string value contained in destination.

Syntax: `strcat(destination, source);`

Example:

```
strcat(message, newmessage );
```

Caution: if the destination array being too small to hold the information, the copy function destroys the content of adjacent memory causing the program to abort, or running with corrupted data.

To avoid the above problem use the `strncat()` function

`Strncat()` function: Copies at most `n` characters of source to destination. If destination has fewer than `n` characters, it pads `sdestination` with `'\0's`.

Syntax: `strncat(destination, source, n)`

String Compare Function

`strcmp()`: Compares `stringExp1` with `stringExp2`. Returns a negative integer if `stringExp1 < stringExp2`, 0 if `stringExp1==stringExp2`, and a positive integer if `stringExp1 > stringExp2`. The comparison is case sensitive.

Syntax: `strcmp(stringExp1, stringExp2);`

Example

```
strcmp(studentName, "Mesfin Girma");
```

`strncmp()`: Compares at most *n characters* of `stringExp1` with `stringExp2`. Returns the same values as `strcmp()` based on the number of characters compared.

Syntax: `strncmp(stringExp1, stringExp2, n);`

Example

```
strcmp(studentName, "Mesfin Girma", 3); //Comparison is based  
only on the first three characters.
```

Strcmpi() is used for case insensitive comparison

C++ string Length Function

`strlen()`: returns the length of the string. Does not include the `'\0'` in the length count.

Syntax: `strlen(str);`

Example:

```
char studentName[20]="Abebe Kebede";
```

```
int size=strlen("Dawit Girma"); //size assigned 11
```

```
int size=strlen(studentName); // ?
```

Example

Write a program that counts the number of occurrence of a word in a text given by the user.

Example

Write a program that determines whether a word is a palindrome.

Length Controlled String

Length Controlled String

- Length controlled string is implemented as a class. The size of the string is stored together with the sequence of characters. The first few bytes are used to store the size.
- The length-controlled string is also called C++-string style.
- For using C++ class string, the program must include a header file `<string>`

C++-string in memory

10	G	i	r	m	a		D	a	w	i	t
----	---	---	---	---	---	--	---	---	---	---	---

Declaration and Initialization of C++ String

For using C++ class string, the program must include a header file known as `<string>`.

Declaration Syntax:

```
string strVar;
```

The declaration for strVar creates a string object of length zero, but the program automatically resizes str1 when it reads input into strVar:

Example:

```
string studentName;
```

Initialization Syntax:

```
string strVar1=literalString;
```

```
string strVar2= strVar1;//strVar1 is prior declared and initialized
```

Example:

```
#include <string>
```

```
string str1="Hello You";
```

```
string str2=str1;
```

Reading and Writing with C++ String

you can use cin with the >> operator to read a one word string object and

cout with the >> operator to display a string object using the same syntax you use with a C-style string.

But reading a line at a time instead of a word at time uses a different syntax. No length specifier is required.

- `getline(cin, str1);` // read to end-of-line
- `getline(cin, str2, ch);` // read to character ch

Operation with C++ String

Unlike C-string, aggregate operation such as assignment, concatenation, appending and comparison is allowed with C++ string.

Assuming the following declaration

```
string str1="Hello There.", str2;
```

Assignment

```
str2=str1; //valid
```

Concatenation

```
str2= str1 + " It is me.";
```

Appending

```
str1= str1 + " It is me.";//equivalent to str1+= " It is me.";
```

Comparison

```
if(str1 > str2) //using relational operator(<, <=, >, >=, ==, !=) is valid  
    ...;
```

Operation with C++ String

C++ also provides different functions to manipulate string. They all can be accessed using the dot notation:

`strVar.functionName`

Assuming the declaration next

`string str1="Hello There. ", str2="It is me.";`

String length function

Returns the length of string.

Syntax: `strVar.size();`

Example: `int size=str1.size(); //size is assigned 11.`

String compare function

Returns positive integer if `strVar1 > strVar2`, negative integer if `strVar1 < strVar2`, zero otherwise.

Syntax: `strVar1.compare(strVar2);`

Example: `if(str1.compare(str2) >0)`

`...;`

Reading and Writing with C++ String

String append function

Appends one string to the end of another string.

Syntax1: `strVar1.append(strVar2);`

Example: `str1.append(str2);` //appends str2 to the end of string1.

Syntax2: `strVar1.append(strVar2, pos2, len2);`

Appends len2 character starting from character at pos2 from strVar2 to strVar1.

Example: `str1.append(str2, 3, 6);` //appends 6 characters starting from character 3 from str2 to the end of str1.

Reading Assignment: Read More on string class functions.

Character by Character Manipulation

Both C and C++ string can be manipulated character by character using index notation.

Example C String

```
char our_string[5] = "Hi";  
int index = 0;  
while (our_string[index] != '\0'){  
    our_string[index] = 'X';  
    index++;  
}
```

Example C ++ String

```
string our_string = "Hi";  
int index = 0;  
while (index <= our_string.size()){  
    our_string[index] = 'X';  
    index++;  
}
```

Array of C++-string

An array of string is represented with C++ as two dimensional array. It is declared as

char arrayName[arraySize][stringSize];

For example an array to hold a maximum of 50 student name is declared as

char name[50][16];

name[0]	A	b	e	b	e		K	e	b	e	d	e	/0			
name[1]	H	i	r	u	t		D	a	w	i	t	/0				
	N	e	b	i		H	a	g	o	s	/0					
	...															
name[20]	M	i	m	i		M	a	m	o	/0						
name[21]	B	r	u	k		K	a	s	a	/0						
	...															
name[48]	T	e	s	f	a	y	e		M	o	g	e	s	/0		
name[49]	B	e	t	i		S	o	l	o	m	o	n	/0			

Array of ...

Because each row represent a single C-string , we can read data in to an array of C-string as follows:

```
char name[50][16];
```

```
//Reading
```

```
for(int j=0; j<50; j++)
```

```
    cin.getline(name[j], n);
```

```
//Printing
```

```
for(int j=0; j<50; j++)
```

```
    cout << name[j] << endl;
```

Note: Look the waste of space when using C-string.

Array of C++ String

An array of string is represented with C++ String as one dimensional array of type string.

It is declared as

```
string arrayName[size];
```

Example

```
string name[50];
```

```
//Reading
```

```
for(int j=0; j<50; j++)
```

```
    getline(cin, name[j]);
```

```
//Printing
```

```
for(int j=0; j<50; j++)
```

```
    cout << name[j] << endl;
```